Open Source Formal Verification

OSFV - Methodology insights

Yann Thoma

Reconfigurable and Embedded Digital Systems Institute
Haute Ecole d'Ingénierie et de Gestion du Canton de Vaud



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License

November 2026

1 Introduction

Planning verification

- When starting a project, start by thinking verification
  - Software engineers have been using Test-driven development (TDD) for years
  - It sometimes helps structure the code
- What methodology
- What language
- What software

Choosing complementary methods

- **Formal:** check properties over modeled behaviors and assumptions, within the selected bound or proof scope; use counterexamples to expose corner cases
- **Simulation:** exercise directed scenarios and constrained-random traffic; inspect detailed waveforms
- **Hardware tests:** validate the integrated device in its real electrical, timing, and system environment

Advantage of formal: Example of an undocumented FIFO requirement

The specification says data is read in order, but does not say what happens when `read` is asserted while the FIFO is empty. Formal analysis exposes a trace where the read pointer advances and later data is skipped. The team clarifies the requirement (ignore the read or report an error), adds a property, and then uses simulation for complementary integration and workload checks.

Simulation

- Language
  - VHDL
  - SystemVerilog
  - Python (cocotb)
- Use random-based scenarios to complement directed testbenches
- Think about a good testbench architecture
  - It should be easy to add new scenarios
- If you wrote assertions, then use them in simulation
- To be realistic, take into account
  - The licenses
  - The competencies in the team
  - A very nice methodology that no one masters is maybe not the best choice

Formal - Starting

- Start with formal as soon as possible
- Design your modules to be *formal-friendly*
  - Separate responsibilities instead of mixing unrelated control and datapath behavior
  - Modules with single responsibilities
  - Think about the elevator example
    - The elevator FSM handles calls, floor/door control, and timer requests; an external counter provides `counter_done_i`
    - Verify the counter and FSM separately, then check their timing contract when integrated
  - Use generics, constants in packages to reduce the space state
- Start with the smallest modules
- Try to reuse your assertions/assumptions for the upper level ones
  - Put reusable interface properties in a vunit bound to the module
  - Keep implementation-specific state checks in a separate vunit

Formal - Implementing

- Write the properties you want to check with proper sentences
- Do not forget the unwritten or hidden specs
  - Challenge yourself with the help of colleagues
- Then write the corresponding PSL code
  - Challenge yourself with the help of colleagues

Formal - Practical workflow

1. Start with local safety properties and a *simple* behaviors/assertions
2. Add only justified environment assumptions; check that they still allow intended behavior
3. Use bounded checks to find bugs; choose the depth to cover the behavior of interest (for example, reset, transaction latency, or counter wrap)
4. For an unbounded claim, use a proof-capable method; expect that complexity may require stronger invariants, decomposition, or simpler properties
5. Run cover checks to see whether important scenarios and property antecedents are reachable

⚠ A bounded PASS means no failure was found up to that depth, not that the property holds for all time

- If a cover goal is unreachable, review reachability, assumptions, and depth before concluding the behavior is impossible

Conclusion

Use formal as much as you can

But do not lose your mind